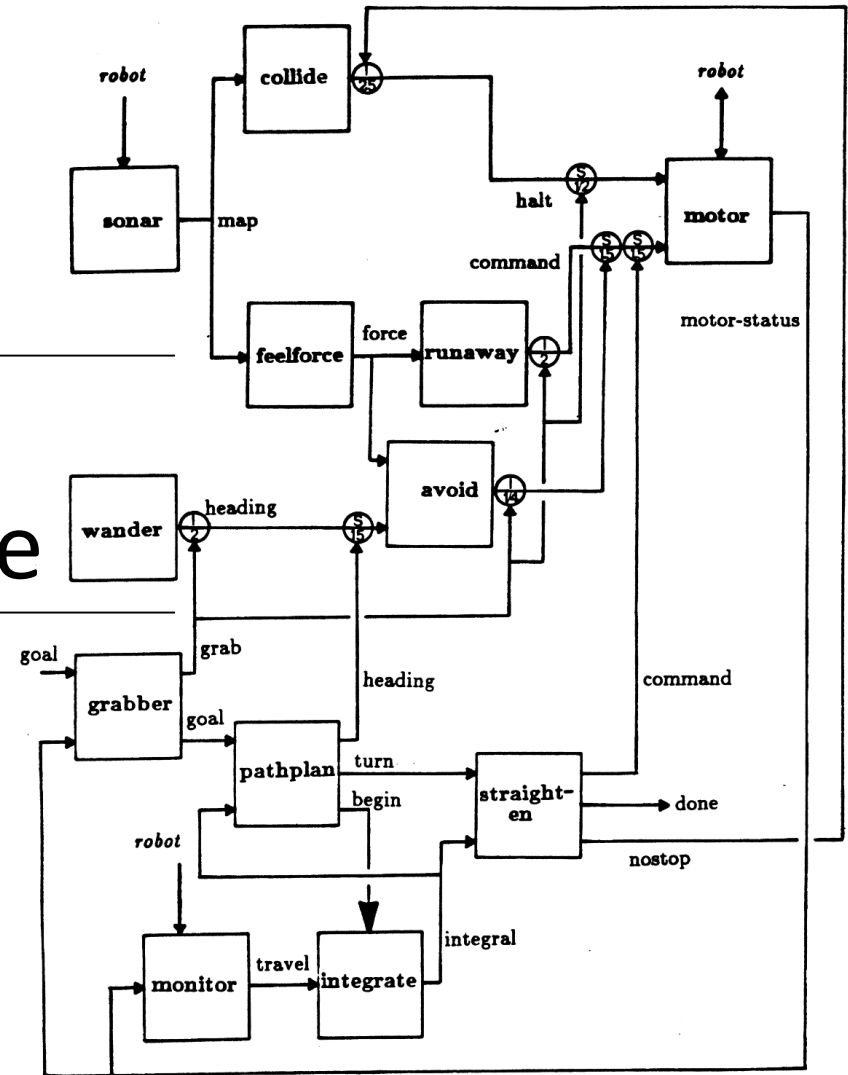


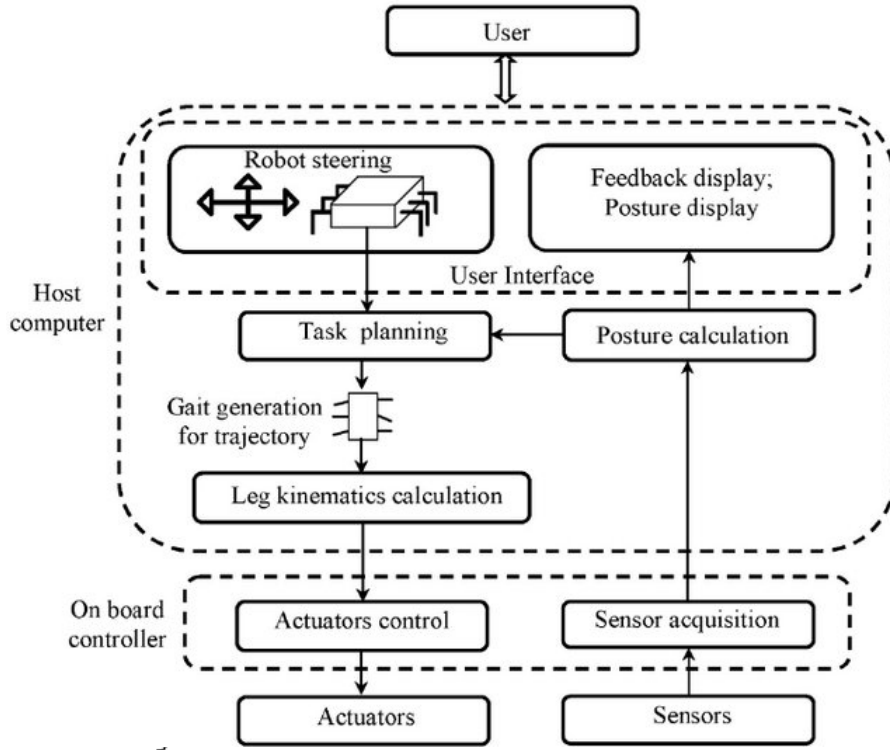
/* Chapitre 2 */ Archi. de contrôle



Brooks, R. (1986). A robust layered control system for a mobile robot. IEEE Journal on Robotics and Automation, 2(1), 14-23.

Couplage perception-action

- Différentes architecture envisageables
- Avec intégration plus ou moins forte de la partie perception et action
- Distinction dans Murphy R. (2000). [An Introduction to AI Robotics](#), MIT Press.



Ségrégation / modularité (par domaine)

- Exemple : déplacement basé vision
- **Perception** : vision par ordinateur
 - Traitement d'images/vidéos
 - Extraction de descripteurs de haut niveau
- **Décision** : intelligence artificielle
 - Méthodes de raisonnement / inférence abstraite
 - Interaction avec utilisateur humain
 - Application de règles / priorités
- **Action** : automatique
 - De la commande aux actionneurs
 - Boucle de contrôle ([ouverte/fermée](#))

A) Couplage quasi-direct

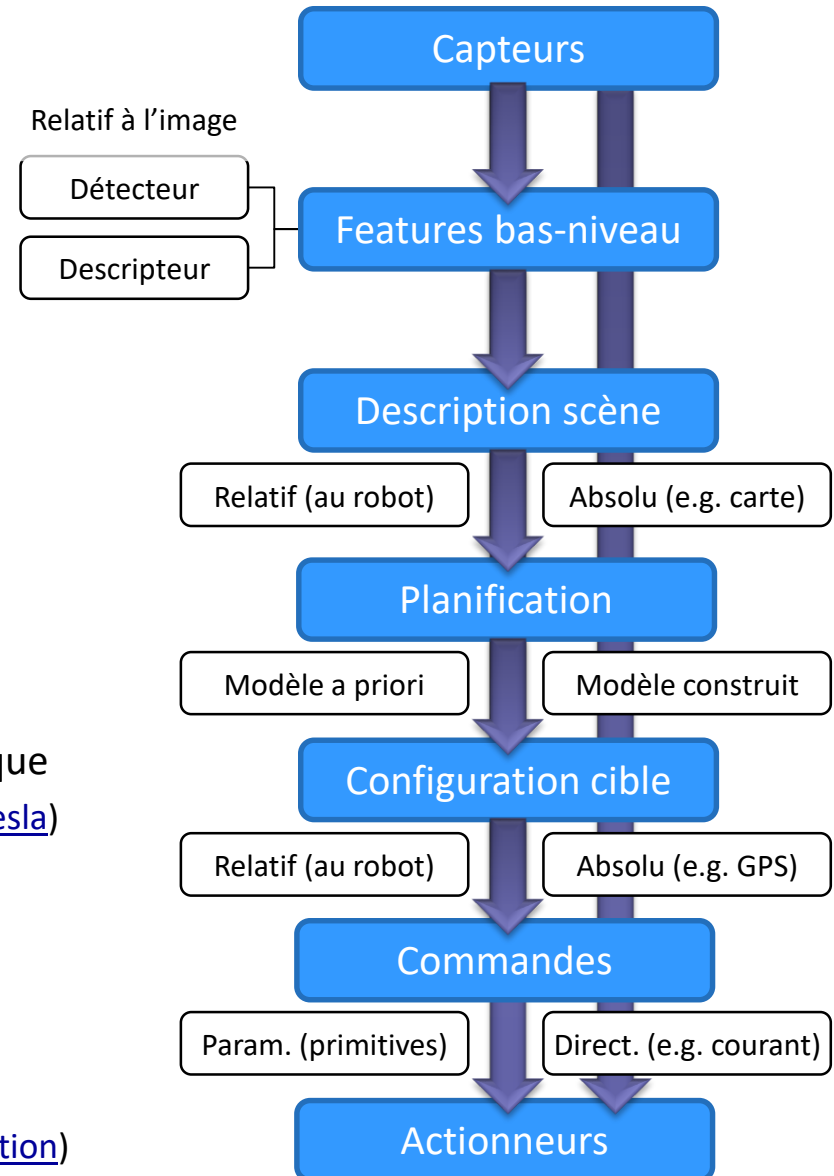
- Véhicules de [Braitenberg](#)
- Systèmes dynamiques
- Contingences sensorimotrices
(e.g., projets européens eSMCs / [socSMCs](#))

B) Architectures hybrides

- Paramétrisation de [primitives motrices](#)
- Asservissement (e.g., [visuel](#))
- Architecture de subsomption

C) Perception – planification – action

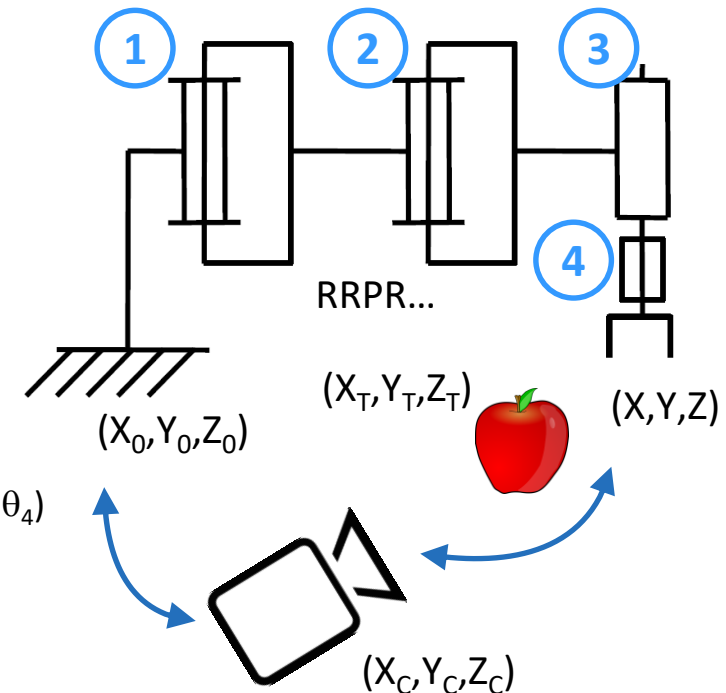
- Méthode de reconstruction / extraction sémantique
 - Deep learning pour reconnaissance visuelle (e.g. [Tesla](#))
 - Reconstruction 3D par ajustement de faisceau
(e.g. CSIRO Zebedee avec [marche](#), [vol](#))
- Architectures cognitiv(ist)es
 - [SOAR](#) ([AGI 2008](#)) / [ACT-R](#) / CLARION / Psi...
- Génération de comportements / trajectoires
 - Pile de tâches (e.g. LAAS, [passage de porte](#), [adaptation](#))



C) Exemple d'un robot de type SCARA + outil

Espaces

- Opérationnel $\rightarrow$ pose de l'outil
 - Position cartésienne de l'outil = coordonnées (X,Y,Z)
 - Orientation de l'outil = angle (θ)
 - Position de la base du robot manipulateur = (X_0,Y_0,Z_0)
- Articulaire $\rightarrow$ configuration du robot
 - Articulations rotoïdes en (1), (2) et (4) = 3 rotations $(\theta_1,\theta_2,\theta_4)$
 - Articulation prismatique en (3) = 1 translation (Δ)



Quelle configuration imposer au robot ?

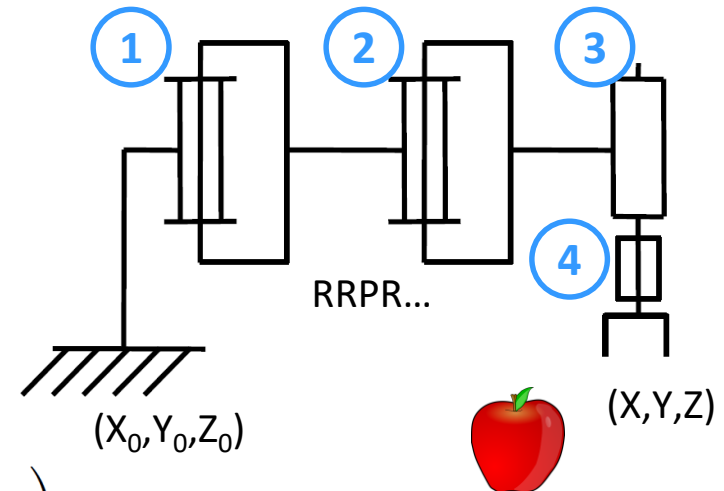
- Où est l'objet à manipuler ?
 - Suppose qu'on le sait, ou qu'on peut le percevoir...
 - Si on doit le percevoir, on doit inférer la position de l'objet à partir des capteurs
- Etapes pour connaître la position de l'objet dans l'espace opérationnel (X_T, Y_T, Z_T)
 - Connaître la position/orientation du capteur (e.g., caméra) dans le repère de la base : (X_C, Y_C, Z_C) et θ_C
 - Pouvoir passer de l'espace caméra à l'espace opérationnel : calibrage + projection + facteur d'échelle
 - Savoir localiser l'objet d'intérêt dans l'image (vision par ordinateur) : coordonnées (U,V) + orientation ?
- Besoin de déterminer la position souhaitée de l'outil (X,Y,Z) à partir de l'objet (X_T, Y_T, Z_T)
- Passer de la connaissance de (X,Y,Z) et (X_0, Y_0, Z_0) à la configuration du robot ?

C) Exemple d'un robot de type SCARA + outil

Transformations matricielles

- 1 articulation = max 3 rotations et/ou 3 translations
- Même si on a simplifié en ne gardant que R et P
- Translation en 3D → vecteur de dimension 3
- Rotation en 3D → matrice de dimension 3x3

$$T = (T_X \ T_Y \ T_Z) \quad R_x(\theta_x) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \theta_x & -\sin \theta_x \\ 0 & \sin \theta_x & \cos \theta_x \end{pmatrix}$$



Coordonnées homogènes

- Pour rendre les calculs homogènes (R ou P), on utilise les coordonnées homogènes (4D) :
 - $P = (X \ Y \ Z \ 1)$ (en théorie on peut multiplier ce vecteur par un facteur w donnant l'échelle)
- Avantage, toute transformation (et donc liaison) peut s'écrire sous la forme d'une matrice H où on réunit P (qui inclut la longueur/position relative du segment) et R (pour les rotations)

$$H = \left(\begin{array}{ccc|c} R_{1,1} & R_{1,2} & R_{1,3} & T_1 \\ R_{2,1} & R_{2,2} & R_{2,3} & T_2 \\ R_{3,1} & R_{3,2} & R_{3,3} & T_3 \\ \hline 0 & 0 & 0 & 1 \end{array} \right)$$

Modèle géométrique direct

- Pour passer de (X_0, Y_0, Z_0) à (X, Y, Z) , il suffit d'enchaîner les transformations : $H_4 \times H_3 \times H_2 \times H_1 \times P_0$

Modèle géométrique inverse

- Direct = on sait comment passer de (X_0, Y_0, Z_0) à (X, Y, Z) si on connaît $(\theta_1, \theta_2, \Delta, \theta_4)$
- Inverse = trouver $(\theta_1, \theta_2, \Delta, \theta_4)$ si on fournit (X, Y, Z) souhaité
- Ca signifie qu'on doit résoudre le système d'équations... et ce n'est pas toujours évident !
 - Méthodes de résolution géométriques, algébriques, numériques selon les cas
 - Dans tous les cas, il y a généralement plein de solutions possibles
- Si on a un robot parallèle, le modèle inverse est simplifié, mais le modèle direct se complique.

Modèle cinématique (direct et inverse)

- En plus de la position (rotation), on contrôle aussi la vitesse
- On utilise la matrice Jacobienne (dérivées de chaque équation du système selon chaque variable)
- Et on retrouve les problèmes de singularités de type 1 et 2

Modèle statique (direct et inverse)

- Pour aussi contrôler les forces et couples appliqués par le robot
- Là aussi, c'est la Jacobienne qui est employée

*Tout ça se complique vite !
(e.g. avec plus de degrés de liberté)*

Modèle dynamique (direct et inverse)

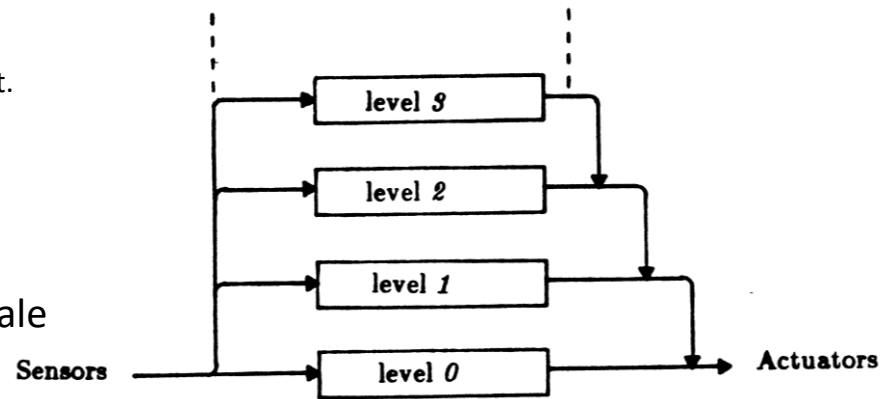
- Pour aussi contrôler le robot en force et moment à tout instant
- Cette fois-ci, on peut utiliser le formalisme de Lagrange (lourd) ou de Newton-Euler (plus rapide)

B) Architecture de subsomption

(Brooks, R. (1986). A robust layered control system for a mobile robot.
IEEE Journal on Robotics and Automation, 2(1), 14-23)

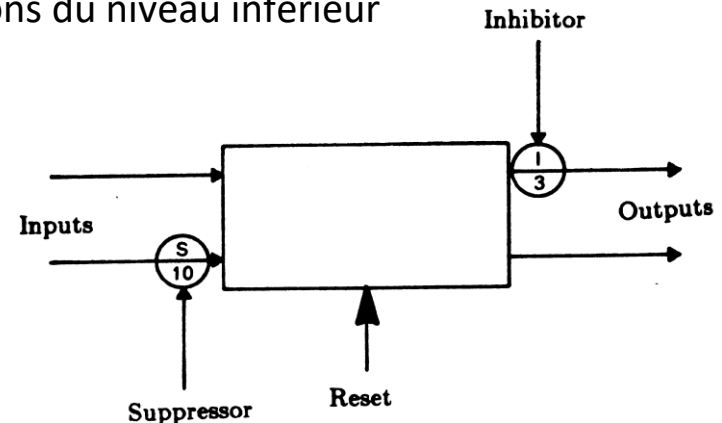
Principes de fonctionnement

- Souci avec architecture hiérarchique / horizontale
= dépendance aux représentations / symboles
- Souci avec architecture réactive / verticale
= conflit potentiel entre les comportements
- Solution proposée par l'architecture de subsomption
= niveau supérieur qui peut « altérer » les informations du niveau inférieur



Structuration

- Définition de niveaux avec priorités différentes
- Niveaux composés de modules (cf. ci-contre)
 - Monitoring des entrées et définitions des sorties
 - Algorithme pour mise à jour en continue
 - Variables internes
- Connexion entre modules via des « inhibiteurs » (sorties) ou « supresseurs » (entrées)



B) Architecture de subsomption

(Brooks, R. (1986). A robust layered control system for a mobile robot. IEEE Journal on Robotics and Automation, 2(1), 14-23)

Exemple de fonctionnement

- Niveau 0
 - **sonar** : perception des distances
 - **motor** : commande au robot
 - **collide** : stop si collision
 - **feelforce** + **runaway** : pour éviter obstacles
- Niveau 1
 - **wander** : exploration de l'environnement
 - **avoid** : prend en compte les obstacles, mais inhibe le comportement de fuite
- Niveau 2
 - **grabber** : inhibition du mouvement afin de stabiliser l'environnement
 - **pathplan** : direction pour atteindre objectif

*Là encore, ça se complique vite !
(e.g. avec comportements moins réactifs)*

